

Agenda – What We Will Cover Today

1. Introduction to Test Case Development
2. Objectives of Test Case Development
3. Understanding Requirements
4. Test Scenario vs Test Case vs Test Script
5. What Makes a Good Test Case
6. Components of a Test Case
7. Types of Test Cases
8. Test Case Design Techniques
9. Test Data Preparation
10. Writing Effective Test Steps
11. Requirement Traceability Matrix (RTM)
12. Test Case Review Process

13. Test Case Prioritization
14. Test Case Management
15. Manual vs Automated Test Cases
16. Real-World Example – Login Functionality
17. Common Mistakes in Test Case Development
18. Best Practices
19. Entry and Exit Criteria
20. Deliverables
21. Test Case Development in Agile
22. Test Automation Readiness
23. Why Strong Test Cases = Better QA Engineers
24. Summary & Q&A

SECTION 01

Introduction to Test Case Development

Foundational concepts and the role of test cases in the STLC

What is Test Case Development?

Definition

Test Case Development is the formal process of creating detailed, structured documents that describe the specific conditions under which a software system will be tested, the steps to be executed, the data to be used, and the expected outcome — all designed to verify that software meets defined requirements.

Role in the Software Testing Life Cycle (STLC)



Why Test Cases Matter in Quality Engineering

- Provide a structured, repeatable approach to verifying software functionality
- Serve as living documentation that captures requirements and expected behavior
- Enable both manual testers and automation frameworks to execute consistent validations
- Create an audit trail from requirements through testing to defect reporting

Impact of Poor Test Case Design on Projects

✓ WELL-WRITTEN TEST CASE

- **TC-001: Verify user login with valid credentials**
- Precondition: User account exists and is active
- Step 1: Navigate to `https://app.example.com/login`
- Step 2: Enter username: 'testuser@qa.com'
- Step 3: Enter password: 'ValidPass@123'
- Step 4: Click 'Login' button
- Expected: Dashboard loads, welcome message displays

✗ POORLY WRITTEN TEST CASE

- **TC-001: Test login**
- Steps: Login to the system
- Expected: It should work
- ✗ No preconditions defined
- ✗ Steps are vague and untestable
- ✗ No test data specified
- ✗ Expected result is ambiguous

Consequences of Poor Test Case Design

Missed Defects

Edge cases and boundary conditions not covered

Wasted Effort

Duplicate, irrelevant, or redundant test execution

Audit Failure

No traceability between requirements and tests

Cost Overrun

Defects found late in production are 10× more expensive

SECTION 02

Objectives of Test Case Development

Why we create test cases — and what they achieve

Objectives of Test Case Development

01

Validate Functionality

Confirm that each feature behaves exactly as the requirement specifies under all defined conditions.

02

Ensure Requirement Coverage

Every functional and non-functional requirement must map to at least one test case, leaving no gaps.

03

Detect Defects Early

Identifying defects during test design (review of test cases) is far cheaper than finding them post-release.

04

Improve Testing Consistency

Structured test cases ensure any tester — junior or senior — executes the same steps and applies the same validation.

05

Support Automation Testing

Well-written manual test cases become direct blueprints for Selenium and Playwright automation scripts.

06

Provide Documentation & Traceability

Test cases form a permanent audit trail connecting business requirements to actual test execution and results.

SECTION 03

Understanding Requirements Before Writing Test Cases

Functional, non-functional, user stories, and acceptance criteria

Types of Requirements a QA Engineer Must Understand

Functional Requirements

- Define WHAT the system should do
- Example: 'The system shall allow users to reset their password via email'
- Directly drive test case creation
- Describe specific behaviors, inputs, and outputs

Non-Functional Requirements

- Define HOW WELL the system performs
- Examples: performance, security, scalability, usability
- Example: 'Page must load within 2 seconds under 1000 concurrent users'
- Drive performance, load, and security test cases

User Stories (Agile)

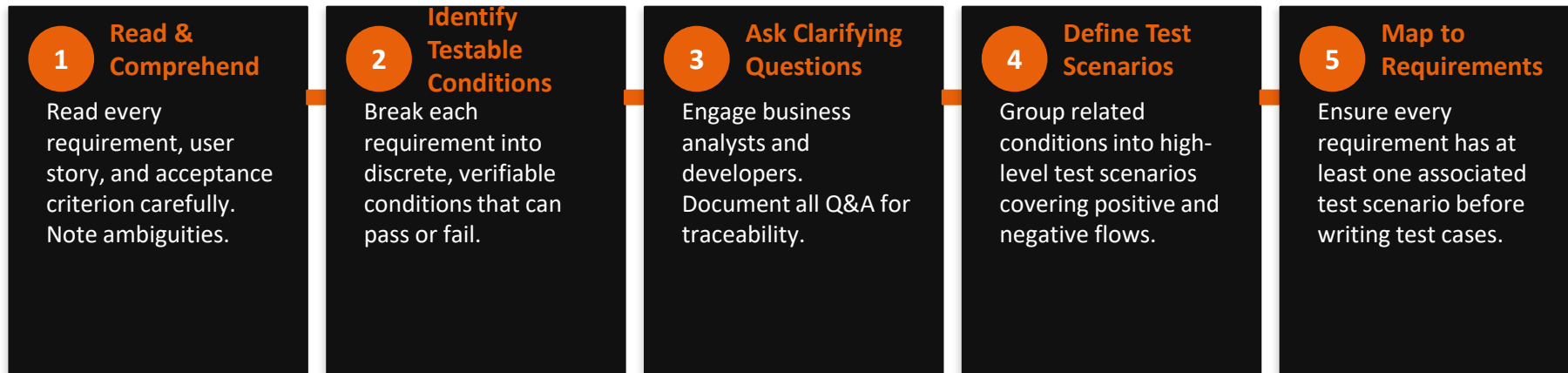
- Written as: As a [role] I want [feature] so that [benefit]
- Example: 'As a registered user, I want to log in with Google SSO so I can access the app faster'
- QA derives test scenarios directly from user stories
- Acceptance criteria within stories become test cases

Acceptance Criteria

- Define the conditions a story must meet to be accepted
- Written as: Given [context] / When [action] / Then [outcome]
- Example: Given user is on login page / When valid credentials entered / Then dashboard loads
- Directly maps to test case expected results

How QA Analyzes Requirements Before Writing Tests

QA Requirement Analysis Process



Real-World Example — E-Commerce Checkout Requirement

REQUIREMENT: 'The system shall allow authenticated users to complete a purchase using a valid credit card.'

- QA Identifies: Login state (authenticated), valid card data, card expiry, CVV, billing address, payment gateway response, order confirmation email
- Test Scenarios Derived: Happy path payment, expired card, insufficient funds, invalid CVV, network timeout during payment, duplicate order prevention
- Acceptance Criteria: Given user is logged in and cart contains items, when valid card details are entered, then order is confirmed and email sent within 2 minutes

SECTION 04

Test Scenario vs Test Case vs Test Script

Understanding the three core artifacts of test case development

Test Scenario vs Test Case vs Test Script — Definitions

Test Scenario

A high-level description of a functionality or business condition to be tested. Captures WHAT needs to be tested without specifying HOW.

EXAMPLE

Scenario: Verify that a registered user can log in to the application.

Test Case

A detailed, step-by-step document specifying the exact conditions, inputs, execution steps, and expected outcomes for validating a specific requirement or scenario.

EXAMPLE

*TC-001: Enter valid username and password
→ Click Login → Verify dashboard loads.*

Test Script

An automated implementation of a test case written in code using a testing framework such as Selenium (Java) or Playwright (TypeScript/Python). Executes the test case programmatically.

EXAMPLE

```
driver.findElement(By.id('username')).sendKeys('user@test.com');  
driver.findElement(By.id('submit')).click();
```

Comparison Table — Test Scenario vs Test Case vs Test Script

Attribute	Test Scenario	Test Case	Test Script
Level of Detail	High-level	Detailed / Step-by-step	Code-level automation
Written By	QA Analyst	QA Analyst / Tester	Automation Engineer
Format	One or two sentences	Structured document / table	Code (.java, .ts, .py)
Tool Used	Jira, Excel, TestRail	TestRail, Excel, Zephyr	Selenium, Playwright, Eclipse, IntelliJ
Reusable?	Partially	Yes	Highly Reusable
Manual Execution?	Requires TC first	Yes — manually	No — automated
Example	<i>Verify user login</i>	<i>Enter valid credentials → Click Login → Verify dashboard</i>	<i>driver.findElement(login).click()</i>

SECTION 05

What Makes a Good Test Case

Characteristics, quality standards, and real examples

Characteristics of a Good Test Case

Clear

Every step and expected result is unambiguous. Any tester can read and execute it with zero interpretation.

Concise

Contains only necessary information. Avoids redundant steps, excessive detail, or narrative-style writing.

Reusable

Designed to be used across multiple test cycles, regression suites, and releases without modification.

Traceable

Every test case links back to a specific requirement, user story, or acceptance criterion in the RTM.

Easy to Execute

Steps are written in logical sequence with clear actions and precise validation points.

Easy to Maintain

When requirements change, the test case can be updated quickly without rewriting the entire suite.

SECTION 06

Components of a Test Case

Every field explained — with a fully completed sample

Components of a Test Case — Field Definitions

Field	Description & Purpose
Test Case ID	Unique identifier (e.g., TC-LOGIN-001). Enables precise referencing in RTM, defect reports, and automation.
Test Scenario	The high-level condition being tested (e.g., 'Verify login with valid credentials').
Test Description	A brief explanation of what the test case verifies and why it is important.
Preconditions	Conditions that MUST be true before the test begins (e.g., 'User account must be active in the database').
Test Steps	Numbered, sequential actions the tester must perform. Each step is one discrete action.
Test Data	Specific input values to be used (e.g., username: 'qa_user@test.com', password: 'Test@1234').
Expected Result	The precise outcome that should occur if the system works correctly.
Actual Result	Filled in during execution — what actually happened. Blank before execution.
Status	Pass / Fail / Blocked / Not Executed. Updated after each test run.
Priority	P1 (Critical) → P2 (High) → P3 (Medium) → P4 (Low). Guides execution order.

Sample Test Case — Login Functionality (Fully Completed)

Field	Value
Test Case ID	TC-LOGIN-001
Test Scenario	Verify successful login with valid credentials
Test Description	This test case verifies that a registered user with valid credentials can successfully log in and is redirected to the dashboard.
Preconditions	1. Application is accessible at https://app.example.com/login 2. Test user account (qa_user@test.com) is active in the database 3. Browser is Chrome 120+
Test Steps	1. Open browser and navigate to https://app.example.com/login 2. Enter 'qa_user@test.com' in the Username field 3. Enter 'Test@1234' in the Password field 4. Click the 'Login' button
Test Data	Username: qa_user@test.com Password: Test@1234 Browser: Chrome 120
Expected Result	User is redirected to the dashboard. Welcome message 'Hello, QA User' is displayed. Session cookie is created.
Actual Result	<i>(To be filled during test execution)</i>
Status	Not Executed
Priority	P1 – Critical

SECTION 07

Types of Test Cases

Functional, UI, positive, negative, boundary, regression, and more

Types of Test Cases — Definitions and Examples

Type	Description	Real-World Example
Functional	Validates that a feature performs its intended function per the requirement.	<i>Verify that a user receives an order confirmation email after successful checkout.</i>
UI / Interface	Validates the appearance, layout, and interactive elements of the user interface.	<i>Verify the 'Login' button is orange, centered, and displays 'Login' text in white 16px font.</i>
Positive	Tests the system with valid, expected inputs to confirm correct behavior.	<i>Enter correct username and password → System logs user in successfully.</i>
Negative	Tests the system with invalid, unexpected, or boundary inputs to ensure graceful handling.	<i>Enter wrong password 5 times → Account is locked and an error message is displayed.</i>
Boundary Value	Tests values at the exact boundary of valid input ranges (min, max, just inside, just outside).	<i>Password field requires 8–20 chars: test with 7, 8, 20, 21 characters.</i>
Regression	Re-executes previously passing test cases after code changes to detect regressions.	<i>After a payment module update, re-run TC-LOGIN-001 through TC-LOGIN-015 to confirm no regressions.</i>
Integration	Validates the interaction between two or more integrated components or systems.	<i>Verify that a completed order in the frontend correctly creates a record in the inventory database.</i>
Smoke	A subset of critical test cases run after every build to confirm basic functionality is not broken.	<i>Verify app launches → User can login → Dashboard loads → Logout works.</i>

SECTION 08

Test Case Design Techniques

Equivalence Partitioning, BVA, Decision Tables, State Transition, Error Guessing

Technique 1: Equivalence Partitioning (EP)

What is it?

Equivalence Partitioning divides input data into equivalent groups (partitions) where all values in a group are expected to behave identically. Instead of testing every value, you test ONE representative value from each partition — reducing the number of test cases while maintaining high coverage.

Example: Age field on a registration form (valid age: 18–65)

Invalid (Below Valid)
Age < 18
Test Value: 15

Test: 15 → Error: 'You must be 18+'

Valid (Accepted Range)
$18 \leq \text{Age} \leq 65$
Test Value: 35

Test: 35 → Registration accepted

Invalid (Above Valid)
Age > 65
Test Value: 70

Test: 70 → Error: 'Age must be 65 or below'

Why EP Improves Coverage

- Reduces the total number of test cases without sacrificing coverage
- Forces testers to think about invalid as well as valid conditions systematically
- Result: 3 well-chosen test cases instead of testing every value from 1–100

Technique 2: Boundary Value Analysis (BVA)

BVA is an extension of Equivalence Partitioning that focuses specifically on the BOUNDARIES of valid and invalid partitions. Most defects occur at boundary values — the transition between valid and invalid. BVA mandates testing the exact minimum, maximum, and values just outside these limits.

Example: Password length must be 8–20 characters

Test Case	Input Value	Boundary Type	Expected Result
TC-BVA-001	7 characters	Just below minimum	Error: Password too short
TC-BVA-002	8 characters	Minimum (valid)	Accepted
TC-BVA-003	9 characters	Just above minimum (valid)	Accepted
TC-BVA-004	14 characters	Middle (valid)	Accepted
TC-BVA-005	19 characters	Just below maximum (valid)	Accepted
TC-BVA-006	20 characters	Maximum (valid)	Accepted
TC-BVA-007	21 characters	Just above maximum	Error: Password too long

Key Insight: The 7 BVA test values cover all critical boundaries — min-1, min, min+1, mid, max-1, max, max+1

Technique 3: Decision Table Testing

Decision Table Testing is used when multiple conditions combine to produce different actions or outcomes. It is ideal for business logic with complex conditional rules — such as pricing, discounts, or access control. Each column in the table represents a unique combination of conditions.

Example: E-Commerce Discount Logic

Condition / Action	Rule 1	Rule 2	Rule 3	Rule 4
Is user a Premium Member?	Yes	Yes	No	No
Order value > R500?	Yes	No	Yes	No
Has promo code?	Yes	No	No	Yes
ACTION: Discount Applied	25%	10%	5%	5%
ACTION: Free Shipping? Result: 4 test cases (one per rule) provide complete coverage of all condition combinations	Yes	No	No	No

- State Transition Testing: Tests transitions between system states based on events (e.g., order status: Pending → Processing → Shipped → Delivered → Cancelled)
- Error Guessing: Based on tester experience, deliberately target areas where defects are most likely (e.g., empty fields, SQL injection, special

SECTION 09

Test Data Preparation

Types of test data, importance, and management strategies

Test Data Preparation — Why It Matters

Test data is the specific set of input values used during test execution. Inadequate test data is one of the most common causes of missed defects. Good test data must cover valid, invalid, boundary, and edge-case scenarios.

Types of Test Data

Valid Data

Data that meets all requirements and should be accepted by the system.

Examples: Username: john.doe@company.com | Password: SecurePass@2024 | Age: 25 | Phone: +27 81 234 5678

Invalid Data

Data that violates requirements and should be rejected with appropriate error messages.

Examples: Username: notanemail | Password: 123 (too short) | Age: -5 or 'abc' | Phone: 123 (too few digits)

Boundary Data

Data at the exact edges of valid ranges, designed to test transitions between valid and invalid.

Examples: Min password: 8 chars exactly | Max username: 50 chars exactly | Age exactly 18 or 65 | Order total exactly R500.00

Negative Data

Data designed to intentionally break the system — special chars, SQL injection, excessively long strings.

Examples: Username: ' OR 1=1 -- | Password: <script>alert(1)</script> | Input: 9999 repeated characters | File: .exe uploaded to image field

SECTION 10

Writing Effective Test Steps

Clarity, precision, and avoiding ambiguity in test step authoring

Good vs Bad Test Steps — With Explanations

Principles of Effective Test Steps

- Action Clarity: Each step describes ONE discrete action (click, enter, navigate, select)
- Validation Points: Steps explicitly state what to verify after each action
- No Ambiguity: Avoid 'fill in the form', 'check the page' — be precise about which field, what value
- Readability: A new tester must be able to execute the test with zero prior knowledge of the feature

✗ BAD TEST STEPS

Step 1: Open the application

- Step 2: Fill in the form
- Step 3: Click the button
- Step 4: Check if it works

Problems:

- 'the application' — which URL?
- 'the form' — which fields? what data?
- 'the button' — which button?
- 'if it works' — what exactly to verify?

✓ GOOD TEST STEPS

- Step 1: Navigate to <https://app.example.com/login>
- Step 2: Enter 'qa_user@test.com' in the 'Email' field
- Step 3: Enter 'Test@1234' in the 'Password' field
- Step 4: Click the orange 'Login' button
- Step 5: Verify the URL changes to /dashboard
- Step 6: Verify 'Welcome, QA User' message is visible
- Step 7: Verify the user avatar appears in the top-right corner

SECTION 11

Requirement Traceability Matrix (RTM)

Mapping requirements to scenarios to test cases — ensuring nothing is missed

Requirement Traceability Matrix (RTM)

The RTM is a document that maps every business requirement to one or more test cases. It ensures complete coverage (no requirement left untested) and enables impact analysis when requirements change. Auditors and clients often require it as proof of thorough testing.

Sample RTM — Login & Password Module

Req. ID	Requirement Description	Test Scenario	Test Case IDs	Status
REQ-001	User can log in with valid credentials	Verify login with valid email and password	TC-001, TC-002	Pass
REQ-002	System locks account after 5 failed attempts	Verify account lockout after failed attempts	TC-003, TC-004, TC-005	Pass
REQ-003	User receives forgot-password reset email	Verify password reset email flow	TC-006, TC-007	Fail
REQ-004	Password must be 8–20 chars with special character	Verify password validation rules	TC-008, TC-009, TC-010, TC-011	Pass
REQ-005	Session expires after 30 minutes of inactivity	Verify session timeout behavior	TC-012, TC-013	In Progress

Key Benefits: Complete coverage visibility | Change impact analysis | Audit compliance | Identifies untested requirements

SECTION 12

Test Case Review Process

Peer reviews, QA lead reviews, and validation against requirements

Test Case Review Process — Steps and Roles

Why Review Test Cases?

Reviewing test cases before execution catches gaps, duplicates, ambiguities, and misalignments with requirements — all at zero cost. A defect in a test case costs more to fix after execution than before.

1. Self-Review

Author (Junior Tester)

Re-read each test case against the requirement it covers. Check all fields are complete. Verify steps are sequential and unambiguous.

2. Peer Review

Another QA Engineer

Independently verify coverage, identify edge cases that were missed, flag duplicate test cases, and confirm test data is realistic.

3. QA Lead Review

QA Lead / Manager

Validate alignment with overall test strategy, confirm priority and severity assignments are correct, check RTM linkage.

4. BA / Dev Review

Business Analyst + Dev

Confirm requirements are correctly interpreted, identify technical constraints or dependencies that affect test cases.

5. Sign-Off

All Stakeholders

Formal approval that test cases are complete, accurate, and ready for execution. Documented sign-off required for audit compliance.

SECTION 13

Test Case Prioritization

Risk-based, business-critical, and frequency-based prioritization

Test Case Prioritization — Frameworks and Examples

Why Prioritize?

Time and resources are always limited. Prioritization ensures the most critical tests run first — so if time runs out, the highest-risk areas are already covered.

P1 – Critical	P2 – High	P3 – Medium	P4 – Low
Core functionality that, if broken, blocks all other testing or makes the system unusable.	Important features that are frequently used and affect the majority of users.	Secondary features that are important but have workarounds if they fail.	Nice-to-have features or cosmetic issues that have minimal business impact.
Examples: • User login / authentication • Payment processing • Application launch / startup	Examples: • Product search / filters • Order placement flow • User profile management	Examples: • Email notifications • Report generation • Export to CSV / PDF	Examples: • Footer link formatting • Tooltip text accuracy • Icon alignment on settings page

SECTION 14

Test Case Management

Organizing, maintaining, and versioning test suites

Test Case Management — Organization and Maintenance

Organizing Test Cases into Suites

- Group test cases by module, feature, or user story (e.g., Login Suite, Payment Suite, Admin Suite)
- Use a hierarchical folder structure in tools like TestRail, Jira Zephyr, or Excel
- Apply consistent naming conventions: [MODULE]-[FEATURE]-[ID] (e.g., PAY-CHECKOUT-003)

Maintaining Test Case Relevance

- Review and update test cases at the start of every sprint or release cycle
- Archive or delete obsolete test cases that no longer map to any active requirement
- Never delete — archive with a reason and date: version control is critical

Version Control Concepts

- Treat test cases like source code — use versioning (v1.0, v1.1, v2.0)
- In tools like Git, test cases in Excel can be committed alongside code
- Change log: record who changed what, when, and why for every revision

Test Suite Execution Strategy

- Smoke Suite: Run on every build (5–10 critical test cases)
- Regression Suite: Run on every release (all passing historical test cases)
- Sprint Suite: Run for new features added in the current sprint only

SECTION 15

Manual vs Automated Test Cases

Converting manual tests into automation with Selenium and Playwright

Manual vs Automated Test Cases — Comparison

Criterion	Manual Test Case	Automated Test Script
Execution Speed	Slow — human must execute step-by-step	Fast — runs in seconds; parallelizable
Best For	Exploratory, UI/UX, ad-hoc, one-time tests	Regression, smoke, repetitive, data-driven
Skills Required	QA knowledge, domain expertise	Programming (Java, TypeScript, Python)
Tools	TestRail, Excel, Jira, Zephyr	Selenium, Playwright, Eclipse, IntelliJ
Maintenance Effort	Low (update a document)	High (update code when UI changes)
Reusability	Low — same effort each execution	High — run thousands of times at no extra cost
Cost Over Time	Increases with each release cycle	Decreases — ROI improves with each run
Suitable When	Requirements frequently change	Requirements are stable and tests run often

Converting Manual Tests to Automation — Selenium & Playwright Examples

Manual Test Case: TC-LOGIN-001 — Verify login with valid credentials



Selenium (Java) — Eclipse / IntelliJ IDEA

```
@Test
public void testValidLogin() {
    WebDriver driver = new ChromeDriver();
    driver.get("https://app.example.com/login");

    driver.findElement(By.id("email"))
        .sendKeys("qa_user@test.com");
    driver.findElement(By.id("password"))
        .sendKeys("Test@1234");
    driver.findElement(By.id("loginBtn"))
        .click();

    String currentUrl = driver.getCurrentUrl();
    Assert.assertTrue(
        currentUrl.contains("/dashboard"),
        "Dashboard not loaded!"
    );
    driver.quit();
}
```



Playwright (TypeScript)

```
import { test, expect } from '@playwright/test';

test('Valid Login Test', async ({ page }) => {
    await page.goto(
        'https://app.example.com/login'
    );

    await page.fill('#email',
        'qa_user@test.com');
    await page.fill('#password',
        'Test@1234');
    await page.click('#loginBtn');

    await expect(page).toHaveURL(
        /.*/dashboard/
    );
    await expect(
        page.locator('.welcome-msg')
    ).toBeVisible();
});
```

SECTION 16

Real-World Example: Login Functionality

Complete walkthrough: Requirement → Scenarios → Test Cases → Data → Defects

Real-World Example — Step 1: The Requirement

REQUIREMENT REQ-LOGIN-001:

'The system shall allow registered users with an active account to log in using their email address and password. Upon successful authentication, the user shall be redirected to the dashboard. If invalid credentials are provided, an appropriate error message shall be displayed without revealing which field is incorrect. The account shall be locked after 5 consecutive failed login attempts.'

QA Analysis — Testable Conditions Identified

1. Valid email + valid password → Dashboard loaded
2. Valid email + INVALID password → Error message (no field-specific hint)
3. INVALID email + valid password → Error message (no field-specific hint)
4. Empty email field → Validation error
5. Empty password field → Validation error
6. 5 consecutive failed attempts → Account locked, lockout notification sent
7. Locked account attempts login → Error: 'Account is locked. Contact support.'
8. Valid login after password reset → Dashboard loaded with reset confirmation banner
9. Session behavior after successful login (cookie, timeout)
10. Password field characters are masked (shown as dots/asterisks)

Real-World Example — Step 2: Test Scenarios & Test Cases

Test Scenarios

TC ID	Test Scenario	Test Data	Expected Result	Type
TC-001	Valid login	Valid email + correct password	Dashboard loads; welcome message shown	Positive
TC-002	Invalid password	Valid email + wrong password	Error: 'Invalid credentials' — no field hint	Negative
TC-003	Invalid email format	'notanemail' + any password	Validation: 'Please enter a valid email'	Negative
TC-004	Empty email	Blank + valid password	Validation error on email field	Negative
TC-005	Empty password	Valid email + blank password	Validation error on password field	Negative
TC-006	Account lockout	Valid email + wrong password ×5	Account locked; lockout email sent	Negative
TC-007	Locked account login	Locked user credentials	Error: 'Account locked. Contact support.'	Negative
TC-008	Password masking	Any valid credentials	Password chars display as '•' during entry	UI
TC-009	Session cookie	Valid login	Session cookie set; expires after 30 min idle	Functional
TC-010	SQL injection	' OR 1=1 -- as password	Login fails; no DB error exposed to user	Security

Real-World Example — Step 3: Possible Defects to Detect

What Defects Could This Login Test Suite Catch?

Defect ID	Severity	Defect Description	Detected By
DEF-001	Blocker	Login button is non-functional — clicking it does nothing	TC-001
DEF-002	Critical	Error message reveals which field is incorrect ('Wrong password') — security vulnerability	TC-002
DEF-003	Critical	SQL injection in password field returns all user data (data breach)	TC-010
DEF-004	Major	Account NOT locked after 5 failed attempts — brute force vulnerability	TC-006
DEF-005	Major	Lockout email not sent to user after account locked	TC-006
DEF-006	Minor	Password characters briefly visible before masking (100ms delay)	TC-008
DEF-007	Minor	Session cookie does not expire after 30 minutes of inactivity	TC-009

Key Insight: A comprehensive test suite of just 10 test cases for Login functionality can detect 7+ critical and major defects — preventing a potential security breach and poor user experience from reaching production.

SECTION 17

Common Mistakes in Test Case Development

What to avoid — with real examples of each mistake

Common Mistakes and How to Avoid Them

✗ Missing Edge Cases

Example: Only testing the 'happy path' login and ignoring empty fields, special characters, and max-length inputs.

✓ Fix: Use BVA and equivalence partitioning systematically for every input field.

✗ Weak Validations

Example: Expected result says 'login should work' instead of 'URL changes to /dashboard and welcome message displays'.

✓ Fix: Specify precise, measurable expected results with exact values, UI states, and messages.

✗ Vague Steps

Example: 'Fill in the login form' — which fields? What data? Which button?

✓ Fix: Each step = one action. Name every field, specify every value, identify every element.

✗ Duplicate Test Cases

Example: TC-005 and TC-012 both test 'Login with correct credentials' with slightly different wording.

✓ Fix: Conduct peer reviews and maintain an RTM to identify and eliminate redundancy.

✗ Lack of Traceability

Example: 100 test cases exist but none are linked to any requirement ID in the RTM.

✓ Fix: Every test case MUST reference a requirement ID. RTM is mandatory, not optional.

✗ No Negative Tests

Example: Only positive test cases written. No testing of what happens when things go wrong.

✓ Fix: For every positive test, write at least one corresponding negative test case.

SECTION 18

Best Practices in Test Case Development

Industry standards for writing, reviewing, and maintaining test cases

Best Practices — Industry Standards

01

Write Reusable Test Cases

Design test cases as modular building blocks. A login test case should be usable in login testing, regression suites, smoke testing, and as a setup step for other tests.

02

Keep Steps Simple and Atomic

Each test step should perform exactly one action. Never combine 'enter username AND password AND click login' into a single step.

03

Ensure Complete Coverage

Use RTM to verify every requirement has at least one test case. Use design techniques (EP, BVA) to ensure all partitions and boundaries are covered.

04

Review Regularly

Test cases are living documents. Schedule quarterly reviews and mandatory reviews at every release cycle. Outdated test cases are worse than no test cases.

05

Align with Requirements

Every test case should have a clear, direct link to a documented requirement or user story. Orphaned test cases waste effort and create confusion.

06

Write Automation-Friendly Test Cases

Use precise locators in your expected results. Identify stable element IDs. Write test data in a structured format that can be externalized for data-driven automation.

SECTION 19

Entry and Exit Criteria

Defining when test case development starts and when it is complete

Entry and Exit Criteria for Test Case Development

ENTRY CRITERIA — When to Start

- ✓ Business Requirements Document (BRD) or User Stories are formally approved and signed off
- ✓ Functional Specification Document (FSD) is available and reviewed by QA
- ✓ Test Planning phase is complete — Test Plan document has been approved
- ✓ Requirement Traceability Matrix (RTM) template is prepared
- ✓ QA team has received a walkthrough of the requirements from BA/Dev
- ✓ Test environment details and test data sources are identified
- ✓ QA team is assembled and responsibilities are assigned

EXIT CRITERIA — When You Are Done

- ✓ Test cases written for 100% of documented requirements
- ✓ Test cases cover all types: positive, negative, boundary, UI
- ✓ All test cases are reviewed and approved by QA Lead
- ✓ RTM is complete with all test cases mapped to requirements
- ✓ Test data document is prepared and reviewed
- ✓ Review comments from BA and Dev have been addressed
- ✓ Test case document is version-controlled and formally signed off

SECTION 20

Deliverables

What the Test Case Development phase produces

Test Case Development Deliverables

Test Case Document

The primary deliverable. Contains all test cases with complete details: ID, scenario, steps, data, expected results, priority, and severity. Stored in TestRail, Excel, or Jira Zephyr.

Format: Excel / TestRail / Zephyr

Requirement Traceability Matrix

Maps every requirement to one or more test cases. Proves complete coverage. Used for audit compliance and change impact analysis.

Format: Excel / Jira

Test Data Document

Specifies all test data sets required for execution: valid data, invalid data, boundary values, and negative data. May include database setup scripts.

Format: Excel / SQL Scripts

Test Case Review Report

Documents the outcome of all review sessions: who reviewed, what was found, what was fixed, and formal sign-off by QA Lead and stakeholders.

Format: Word / PDF / Email

SECTION 21

Test Case Development in Agile

Writing test cases during sprints, working with user stories, and team collaboration

Test Case Development in an Agile Environment

In Agile projects, test case development is NOT a separate phase — it happens concurrently with development within each sprint. QA collaborates continuously with developers and product owners to write, review, and update test cases as requirements evolve.

Agile Test Case Development Workflow (Within a Sprint)

Day 1–2	Day 2–3	Day 3–4	Day 5–8	Day 9–10
Sprint Planning QA reviews user stories, acceptance criteria, and definition of done. Identifies test scenarios for the sprint backlog.	Test Case Writing QA writes test cases for sprint stories. Developer builds the feature in parallel. QA flags ambiguities immediately.	Three Amigos BA, Dev, and QA meet to review acceptance criteria and test cases together. Misunderstandings caught early.	Execution & Review QA executes test cases as features are built. Defects logged in Jira. Test cases updated based on code changes.	Regression & Closure Regression suite runs. Remaining defects assessed. Test cases finalized and sprint retrospective captures lessons.

Key Agile Practices:

- User stories serve as the primary source for test case derivation
- Test cases are reviewed in the Three Amigos meeting (BA + Dev + QA)
- Acceptance criteria in user stories directly become test case expected results

SECTION 22

Test Automation Readiness

What makes a test case automation-friendly — Selenium & Playwright

What Makes a Test Case Automation-Ready?

Characteristics of Automation-Ready Test Cases

Stable, Unique Locators

Expected results must reference stable element IDs, data-testid attributes, or ARIA roles — NOT fragile XPaths based on visual structure.

```
Selenium: By.id('loginBtn') | Playwright:  
page.locator('[data-testid=login-btn]')
```

Precise Expected Results

Exact text, URL patterns, element visibility states, and HTTP response codes — not vague descriptions like 'page should load'.

```
Assert.assertEquals(driver.getTitle(), 'Dashboard |  
App Name')
```

Externalized Test Data

Test data must not be hardcoded in test steps. Use data-driven design with external data sources (Excel, CSV, JSON).

```
@DataProvider in TestNG | test.each() in  
Playwright
```

Independent Test Cases

Each test case must be executable independently without relying on the state left by a previous test. Use @BeforeEach / @AfterEach setup and teardown.

```
@BeforeMethod / @AfterMethod (TestNG) |  
beforeEach() / afterEach() (Playwright)
```

Reusable Steps

Common actions (login, navigate, logout) should be written once as reusable methods in a Page Object Model and called from multiple test cases.

```
Page Object Model (POM) – LoginPage.java /  
LoginPage.ts
```

SECTION 23

Why Strong Test Case Development Makes Better QA Engineers

The skills you build and why they matter for your career

How Test Case Development Builds QA Excellence

1

Analytical Thinking

Breaking requirements into discrete, testable conditions trains your mind to think systematically and spot edge cases that others miss.

You become the engineer who catches defects before users ever see them.

2

Requirement Understanding

Deep analysis of requirements to write accurate test cases forces you to truly understand the business domain — not just test buttons.

You become a bridge between business and technology.

3

Defect Detection

Well-designed test cases covering positive, negative, boundary, and edge cases maximize the number of defects caught in testing.

*Higher defect detection = lower production incidents
= happier clients.*

4

Automation Readiness

Writing clear, precise, automation-friendly test cases accelerates the transition to Selenium and Playwright automation.

Manual → Automation skills progression = career advancement.

5

Communication Skills

Test cases are read by developers, managers, clients, and auditors. Clear, professional test case writing demonstrates domain expertise.

QA engineers who write well are trusted, promoted, and valued.

SECTION 24

Summary

Key takeaways from Test Case Development

Key Takeaways — Test Case Development

1

Test Case Development is not documentation — it is quality engineering: a structured discipline that determines what gets tested, how it is tested, and what constitutes a pass or fail.

2

Understand requirements deeply before writing a single test case. Ask questions, challenge ambiguities, and document everything.

3

Use design techniques (Equivalence Partitioning, BVA, Decision Tables) to achieve maximum coverage with minimum redundancy.

4

A good test case is clear, concise, reusable, traceable, executable, and maintainable — it is written for anyone to execute, not just the author.

5

Every test case must link to a requirement in the RTM. If it doesn't link to a requirement, question why it exists.

6

Negative, boundary, and edge-case testing is as important as positive testing — most production defects are found at the boundaries.

7

Manual test cases are blueprints for automation. Writing precise test cases now makes Selenium and Playwright automation faster and more reliable.

8

Strong test case development skills make you a better QA engineer: sharper analyst, stronger communicator, and more automation-ready professional.